

Contents

guide_DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning 1

0. 这篇论文到底改变了什么 1

1. 回到 2025 年的语境 2

2. 原论文阅读地图 2

3. R1-Zero 方法图 3

4. GRPO 和 PPO 的关系 3

5. GRPO loss 拆开看 4

6. Rule-Based Reward 5

7. R1-Zero 的训练现象 5

8. 为什么 R1-Zero 还不够 6

9. DeepSeek-R1 多阶段 pipeline 6

10. R1 各阶段证据链 7

11. Test-Time Scaling 7

12. Distillation 的意义 7

13. 与本仓库代码怎么对上 8

14. 极简 GRPO 数字例子 8

15. 极简代码: rule reward + group advantage 9

16. 理论和动机上的深点 9

17. 局限 9

18. 常见误区 10

19. 闭卷掌握检查 10

20. 用 AI agent 学这篇的正确方式 10

guide_DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

原论文: DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

本地原文 PDF: learning/reasoning-r1/paper/01_deepseek_r1.pdf

作者: DeepSeek-AI

年份: 2025

类型: technical report / paper

0. 这篇论文到底改变了什么

DeepSeek-R1 的核心不是一个新 Transformer 结构, 而是一套 reasoning post-training 范式。它展示了一个非常重要的 claim:

在可验证任务上, LLM 的推理能力可以通过大规模 RL 被激励出来, 不一定先依赖人类标注的完整 reasoning trajectories。

论文有两条主线:

DeepSeek-R1-Zero:
base model

-> rule-based reward
-> GRPO
-> long CoT / reflection / verification emerge

DeepSeek-R1:

cold-start reasoning data
-> RL
-> rejection sampling + SFT
-> second RL with reasoning/general rewards
-> stronger reasoning + better readability + broader instruction following

读这篇时要抓住一个平衡: R1-Zero 证明了“纯 RL + 可验证奖励”可以诱发推理行为, 但它也暴露了可读性差、语言混杂、通用任务弱等问题。R1 的多阶段 pipeline 则说明实际可用模型不能只靠纯 RL, SFT、rejection sampling、reward models、language consistency reward 和蒸馏都很重要。

1. 回到 2025 年的语境

在 R1 之前, 推理能力通常依赖几类方法:

- Chain-of-thought prompting: 给 few-shot reasoning examples, 或者提示 “think step by step”。
- SFT on reasoning traces: 用人工或强模型写好的多步推理轨迹训练模型。
- RLHF / PPO: 用人类偏好或 reward model 优化 assistant 行为。
- Test-time scaling: 多采样、majority voting、tree search、verifier reranking。

这些路线的问题是: 人类标注 reasoning traces 昂贵, 而且可能把模型限制在“人类示范过的推理模式”里。R1-Zero 的激进选择是跳过传统 SFT, 让 base model 在可验证任务上自己探索。

这不是说 SFT 没用。论文后半部分反而强调: 对 open-ended QA、creative writing、general instruction following 这类难以定义可靠 reward 的任务, SFT 仍然不可替代。R1 的观点更像是:

能用可靠 verifier 的地方, 让 RL 放开探索;
不能可靠验证的地方, 用 SFT 和偏好奖励补足行为质量。

2. 原论文阅读地图

建议按下面顺序读:

1. Abstract/Introduction: 看 “reasoning can be incentivized through pure RL” 的主张。
2. Section 2.1: 看 GRPO, 尤其是 group advantage 和 reference KL。
3. Section 2.2: 看 rule-based reward, accuracy reward + format reward。
4. Figure 1: 看 R1-Zero AIME accuracy 和 response length 在训练中的增长。
5. Table 2: 看 “aha moment”, 理解 reflection/rethinking 如何出现。
6. Section 3/Figure 2: 看 DeepSeek-R1 多阶段 pipeline。
7. Section 3.2: 看第一阶段 RL、第二阶段 RL、language consistency reward。
8. Table 3: 看各阶段在 reasoning、coding、general preference 上的变化。
9. Section 4/Limitations: 看 tool use、token efficiency、language mixing、prompt sensitivity、reward hacking。
10. Appendix F: 看 distillation 与 RL 的比较。

如果只读一次, 必须把 R1-Zero 和 R1 分开。R1-Zero 是科学实验味更重的 “纯 RL 能否长出推理”; R1 是工程模型训练 pipeline。

3. R1-Zero 方法图

```
DeepSeek-V3-Base
|
| no SFT before RL
v
sample G responses per question
|
| rule reward:
|   accuracy reward
|   format reward
v
group-relative advantages
|
| GRPO update:
|   PPO-style ratio clipping
|   KL to reference policy
|   no learned value model
v
DeepSeek-R1-Zero
|
| emergent:
|   longer CoT
|   self-verification
|   reflection
|   alternative solution search
```

它故意只给结构约束，不教具体思考内容。论文使用 template 要求模型先生成 reasoning process，再给 final answer；但真正的 reasoning pattern 由 RL 训练过程探索出来。

4. GRPO 和 PPO 的关系

PPO 需要 value model 来估计 advantage:

$$A_t = \text{return}_t - V(s_t)$$

在 long CoT reasoning 里，这很难。因为只有最终答案有可靠 reward，中间 token 是否“好”很难判断；而且模型可能先走错路，再反思修正。让 value model 根据部分 response 预测最终 reward，难度很高，成本也大。

GRPO 的改法是：对每个 question 同时采样一组 outputs，用组内 reward 做 baseline。

```
question q
-> sample outputs o_1, o_2, ..., o_G
-> compute rewards r_1, r_2, ..., r_G
-> group advantage:
```

$$A_i = (r_i - \text{mean}(r_1 \dots r_G)) / \text{std}(r_1 \dots r_G)$$

这样就不需要训练 value model。某个回答不是绝对“好”或“坏”，而是相对于同一个 prompt 下的其它回答更好或更差。

本仓库的最小实现:

```
def compute_group_advantage(rewards, k):
    rewards = rewards.reshape(-1, k)
    mean = rewards.mean(dim=1, keepdim=True)
    std = rewards.std(dim=1, keepdim=True) + 1e-8
    return ((rewards - mean) / std).reshape(-1)
```

张量级别:

B = number of questions
 G = responses per question
 T = response token length

```
rewards:          [B*G]
rewards_group:    [B, G]
advantages:       [B*G]
logp_old:         [B*G, T]
logp_new:         [B*G, T]
logp_ref:         [B*G, T]
response_mask:    [B*G, T]
```

5. GRPO loss 拆开看

GRPO 仍然保留 PPO 风格的 ratio clipping:

```
ratio = pi_new(o_i | q) / pi_old(o_i | q)
```

```
surrogate =
    min(
        ratio * A_i,
        clip(ratio, 1-eps, 1+eps) * A_i
    )
```

同时加入 reference policy 的 KL:

```
loss = - surrogate + beta * KL(pi_new || pi_ref)
```

本地代码对应:

```
ratio = (log_probs_new - log_probs_old).exp()
A = advantages.unsqueeze(1)
surr1 = ratio * A
surr2 = ratio.clamp(1 - eps, 1 + eps) * A
L_clip = -torch.min(surr1, surr2)
```

```
log_r = log_probs_ref - log_probs_new
kl = log_r.exp() - log_r - 1
```

```
loss = masked_mean(L_clip) + beta * masked_mean(kl)
```

这里的 KL estimator 是 Schulman 的非负近似形式。重要区别:

- PPO-RLHF 常把 per-token KL 作为 dense reward 加进 reward。
- GRPO 论文实现把 KL estimator 直接放到 loss 里。

论文认为这对 long CoT 有意义, 因为 PPO 的 dense KL reward 会惩罚累计 KL, 可能间接压制 response length 的增长。R1 训练又需要模型能 “多想一会儿”, 所以不能让 KL 机制过度抑制长推理。

6. Rule-Based Reward

R1-Zero 的 reward 主要是两类:

```
rule_reward = accuracy_reward + format_reward
```

Accuracy reward:

- 数学题: final answer 可和 ground truth 比较。
- 代码竞赛: 可用 compiler / tests 检查。
- 逻辑推理: 在可验证数据上判定结果是否正确。

Format reward:

- 要求 reasoning 和 answer 使用指定结构。
- 例如用 <think>...</think> 包住思考, 用 answer 部分给最终答案。

本地代码对应:

```
def format_reward(response):  
    ok = response matches "<think>...</think><answer>...</answer>"  
    return 1.0 if ok else 0.0
```

Countdown/GSM8K 类 accuracy reward:

```
def countdown_reward(predicted, numbers, target):  
    # check the expression uses the provided numbers  
    # safely evaluate whether it equals target  
    return 1.0 or 0.0
```

论文刻意不在 reasoning tasks 上使用 neural reward model 或 PRM。理由是大规模 RL 下 neural RM 容易被 reward hacking, 而且反复重训 RM 会增加复杂度和算力成本。这个选择是 R1-Zero 成功的关键前提: reward 必须可靠。

7. R1-Zero 的训练现象

原文报告 DeepSeek-R1-Zero 在 AIME 2024 上:

- 初始 average pass@1: 15.6%。
- RL 后 average pass@1: 77.9%。
- 加 self-consistency decoding 后: 86.7%。

更重要的不是单个数字, 而是训练过程中出现的行为变化:

- response length 稳定增长。
- 模型开始生成更长的 chain-of-thought。
- 出现 self-reflection。
- 出现 verification。
- 出现探索 alternative approaches。
- 论文展示了 “wait, wait” 式的 aha moment, 模型会停下来重新检查自己的推导。

这就是论文所谓 self-evolution 的证据链: 没有人显式教模型 “遇到错误要反思”, 但在可验证奖励的压力下, 反思和重算成为能提高正确率的策略。

8. 为什么 R1-Zero 还不够

R1-Zero 的问题也很明确:

1. 可读性差: 长 CoT 可能凌乱。
2. 语言混杂: 可能在中英文之间混用, 尤其在思考链里。
3. 任务覆盖窄: rule-based RL 主要针对数学、代码、逻辑等可验证任务。
4. 通用写作、开放问答、用户偏好对齐较弱。
5. 纯 RL 依赖可靠 reward, 开放任务难以构造。

这就是为什么 DeepSeek-R1 不是简单 “继续 R1-Zero 训更久”, 而是引入 cold-start data、rejection sampling、SFT、general data 和第二阶段 RL。

9. DeepSeek-R1 多阶段 pipeline

Stage 0: Base

DeepSeek-V3-Base

Stage 1: Cold start SFT

thousands of high-quality long CoT examples

goal: readable, conversational, human-aligned thinking format

Stage 2: Reasoning RL

GRPO

rule-based rewards for reasoning

language consistency reward

Stage 3: Rejection sampling + SFT

collect high-quality reasoning samples from current model

mix with non-reasoning data

improve reasoning plus writing/general instruction ability

Stage 4: Second RL

reasoning rewards for verifiable tasks

reward models for general/helpfulness/safety data

final DeepSeek-R1

这条 pipeline 的思想是:

RL discovers stronger reasoning trajectories.

SFT makes behavior readable and broad.

Second RL aligns final behavior with both reasoning and user preferences.

论文第一阶段 RL 的一些关键设置:

- learning rate: $3e-6$ 。
- KL coefficient: 0.001。
- rollout temperature: 1。
- 每个 question 采样 16 个 outputs。
- maximum length: 32,768。
- 每步 32 个 unique questions, 所以 batch size 为 512。
- 每 400 steps 用最新 policy 替换 reference model。
- 引入 language consistency reward 缓解语言混杂。

第二阶段 RL 主要保留第一阶段参数，但降低 temperature 到 0.7，并混入 reasoning/general reward signals。

10. R1 各阶段证据链

Table 3 给出阶段变化，读的时候不要只看最终 R1，要看每一步的功能：

- R1-Zero: reasoning 很强, AIME 2024 为 77.9%, MATH-500 为 95.9%, 但 instruction following 和 user preference benchmark 弱。
- R1-Dev1: 加 cold-start 后 IF-Eval 和 ArenaHard 大幅改善，但 AIME 下降到 59.0%，说明少量 cold-start 数据改善可读性同时可能损伤一部分 reasoning。
- R1-Dev2: 经过 reasoning RL 后，AIME 回到 74.0%，coding/math/STEM 增强。
- R1-Dev3: 加 reasoning + non-reasoning datasets 做 SFT，AlpacaEval 和 Aider-Polyglot 改善。
- Final R1: 混合 reasoning/general RL 后，AIME 2024 79.8%，MATH-500 97.3%，Codeforces percentile 96.3，ArenaHard 92.3。

论文自己的解释是：reasoning-oriented RL 主要提升推理、代码、数学和 STEM；general preference benchmark 的大幅提升主要来自后续 SFT 和最终混合 RL。

这点很关键。不要把所有提升都归因于“纯 RL”。R1 的最终质量来自多阶段组合。

11. Test-Time Scaling

R1 的一个重要现象是 adaptive thinking length：

- 简单题用较少 thinking tokens。
- 难题用更多 thinking tokens。
- 原文在 2024 数学竞赛题集会上报告，DeepSeek-R1 平均使用 8,793 thinking tokens，简单题少于 7,000，难题超过 18,000。

这和 majority voting 不一样。Majority voting 是多个样本彼此独立，不能在同一条推理链里 backtrack/self-correct。R1 的长 CoT 允许模型在一个样本内检查、反思和修正。

但论文也承认，长推理并不总是成功。AIME 2024 上 R1 pass@1 是 79.8%，pass@64 是 90.0%；majority voting 可把 R1 从 79.8% 提到 86.7%。这说明：

```
long CoT improves single-sample reasoning,  
but independent sampling / voting can still complement it.
```

12. Distillation 的意义

R1 还做了大量 distillation：

```
DeepSeek-R1 generates 800,000 samples  
open-source base models receive SFT on those samples  
no RL stage for distilled models in this paper
```

论文报告 distilled smaller models 很强，并强调：

- Distill-Qwen-1.5B 就能在数学 benchmark 上超过非推理 baseline。
- 学生模型越大，蒸馏效果越强。
- 对小模型来说，从强 teacher 蒸馏通常比直接做大规模 RL 更经济、更有效。
- 但要突破 teacher 边界，仍可能需要强 base model 和更大规模 RL。

Appendix F 的结论尤其重要: 32B base model 做大规模 RL 可以达到不错水平, 但 DeepSeek-R1-Distill-Qwen-32B 在多个 benchmark 上显著强于 Qwen2.5-32B-Zero。也就是说, distillation 是传播推理能力的高效方式; pure RL from scratch 更依赖 base model 能力和算力。

13. 与本仓库代码怎么对上

本模块代码正好对应论文关键部件:

- `src/grpo_minimal.py`: GRPO group advantage、PPO-style clip、reference KL。
- `src/rewards/format_reward.py`: 检查 `<think>...</think><answer>...</answer>` 格式。
- `src/rewards/accuracy_reward.py`: GSM8K / Countdown 可验证答案奖励。
- `src/r1_zero_track_a.py`: Countdown-3 toy pipeline, 模拟 R1-Zero 的 rule reward + GRPO。
- `src/rloo_minimal.py`: RLOO baseline, 帮助比较 group baseline 变体。
- `src/reinforce_pp.py`: REINFORCE++ 风格简化, 展示 critic-free PPO-like update。

学习时建议先跑组件 smoke test:

```
python learning/reasoning-r1/src/grpo_minimal.py
```

你要观察:

- group advantage 每组均值接近 0。
- reward 形状从 `[B*G]` reshape 成 `[B,G]`。
- advantage broadcast 到每个 response token。
- loss 只在 response mask 上平均。
- KL estimator 是否非负。

14. 极简 GRPO 数字例子

假设同一个 prompt 采样 4 个回答, reward 为:

```
r = [1, 0, 1, 0]
mean = 0.5
std ~= 0.577
```

Group advantages:

```
A ~= [(1-0.5)/0.577, (0-0.5)/0.577,
      (1-0.5)/0.577, (0-0.5)/0.577]
     ~= [0.866, -0.866, 0.866, -0.866]
```

含义:

- 两个正确回答被推高。
- 两个错误回答被压低。
- 这个 baseline 只比较同一个 prompt 内的回答, 减少 prompt 难度差异带来的噪声。

如果另一个 prompt 四个回答全错:

```
r = [0, 0, 0, 0]
std = 0
```

这时 advantage 没有有用学习信号。实际系统需要足够大的 group、足够好的 sampling 和足够多可验证问题, 才能让正样本出现。

15. 极简代码: rule reward + group advantage

```
import torch

def group_advantage(rewards, k):
    rewards = rewards.reshape(-1, k)
    mean = rewards.mean(dim=1, keepdim=True)
    std = rewards.std(dim=1, keepdim=True) + 1e-8
    return ((rewards - mean) / std).reshape(-1)

rewards = torch.tensor([1.0, 0.0, 1.0, 0.0])
A = group_advantage(rewards, k=4)
print(A)
```

简化的 GRPO token loss:

```
ratio = torch.exp(logp_new - logp_old)
A_tok = A.unsqueeze(1)
surr1 = ratio * A_tok
surr2 = ratio.clamp(1 - eps, 1 + eps) * A_tok
policy_loss = -torch.min(surr1, surr2)

kl = torch.exp(logp_ref - logp_new) - (logp_ref - logp_new) - 1
loss = masked_mean(policy_loss) + beta * masked_mean(kl)
```

这段代码就是 R1-Zero 训练直觉的骨架: 对同题多答案比较, 正确答案更容易得到正 advantage, 错误答案得到负 advantage。

16. 理论和动机上的深点

R1 论文最值得琢磨的是“为什么跳过 SFT 可能有帮助”。作者的假设是: 人类写的 reasoning traces 可能不完整, 也可能带有认知偏见。SFT 会让模型模仿这些轨迹, 但不一定鼓励模型搜索更好的轨迹。

RL 的作用不是让模型“学会人类写过的步骤”, 而是在可验证 reward 下让模型自己发现提高正确率的行为。长 CoT、反思、重新检查、换策略, 都是为了提高最终答案正确率而出现的工具性行为。

但这只在 reward 可靠时成立。数学/代码有明确 verifier, 因此可以扩大 RL。写作/开放问答没有明确正确答案, 纯 RL 容易学会 reward hacking 或奇怪行为。这就是论文最后强调 iterative pipeline 的原因:

RL expands reasoning exploration where verifier is reliable.
SFT shapes behavior where reward is ambiguous.

17. 局限

论文列出的局限包括:

1. Structured output 和 tool use 仍弱, 不能使用搜索、计算器等外部工具。
2. Token efficiency 仍有改进空间, 简单问题可能 overthinking。
3. Language mixing 对非中英文 query 可能更明显。
4. Prompt sensitivity: few-shot prompting 反而可能降低性能, 建议 zero-shot 明确描述问题和输出格式。
5. Software engineering benchmark 提升有限, 因为评测慢, 难以进行大规模 RL。
6. Reward hacking: 纯 RL 成功依赖可靠 reward, 开放任务 reward 难定义。
7. Small model direct RL 效果依赖 base model capacity, 蒸馏往往更经济。

这些局限告诉我们: R1 不是 “RL 解决一切”。它更像是证明了一个方向: 如果任务可验证、base model 够强、算力足够, RL 可以把潜在推理能力激发出来。

18. 常见误区

1. 误区: R1 完全没有 SFT。
 - 更准确: R1-Zero 跳过 SFT 直接 RL; 最终 R1 使用 cold-start SFT、rejection sampling、SFT 和多阶段 RL。
2. 误区: R1 只靠格式奖励。
 - 更准确: 关键是 accuracy reward; format reward 只是结构约束。
3. 误区: GRPO 就是 PPO 改个名字。
 - 更准确: GRPO 用 group reward baseline 替代 learned value model, 并改变 KL 的使用方式。
4. 误区: Aha moment 是人为写进去的。
 - 更准确: 论文展示它是训练中出现的反思行为, 但这个现象应理解为 reward-driven behavior, 不要神秘化。
5. 误区: 小模型也可以直接 RL 出 R1 级推理。
 - 更准确: 论文强调 pure RL from base strongly depends on base model capacity; 小模型常常更适合从强 teacher 蒸馏。

19. 闭卷掌握检查

1. R1-Zero 和 R1 的训练 pipeline 有什么区别?
2. 为什么 R1-Zero 故意跳过 SFT?
3. GRPO 的 group advantage 公式是什么?
4. GRPO 为什么可以不训练 value model?
5. Accuracy reward 和 format reward 分别解决什么?
6. 为什么论文避免在 reasoning tasks 上使用 neural reward model?
7. R1-Zero 的 AIME 2024 pass@1 从多少提升到多少?
8. Aha moment 在论文里代表什么训练现象?
9. Cold-start data 解决 R1-Zero 的哪些问题?
10. R1 final 的提升为什么不能全部归因于纯 RL?
11. Distillation 和 direct RL 对小模型的 trade-off 是什么?
12. 为什么 reliable verifier 是 R1 这条路线的核心瓶颈?

20. 用 AI agent 学这篇的正确方式

不要让 agent 只总结 “R1 使用 GRPO”。更好的 prompt 是:

我正在读 DeepSeek-R1。请你先让我区分 R1-Zero 和 R1。

然后用 B=2, G=4, T=6 的例子带我画出 GRPO 的 rewards、advantages、logprobs 和 mask shape。

接着让我手算 rewards=[1,0,1,0] 的 group advantage。

然后问我 accuracy reward、format reward、language consistency reward 分别服务哪一段 pipeline。

最后用论文数字检查证据链: AIME 15.6->77.9, self-consistency 86.7, final R1 AIME 79.8。

如果我说 “R1 证明纯 RL 适合所有任务”, 请纠正并解释 verifier 和 reward hacking 的边界。

真正掌握这篇论文的标志是: 你能把 R1-Zero 的纯 RL 实验和 R1 的工程 pipeline 分开; 能手算 GRPO group advantage; 能解释为什么可靠规则奖励能诱发反思和长 CoT; 也能指出纯 RL 的边界、蒸馏的经济性和最终模型质量来自多阶段组合。